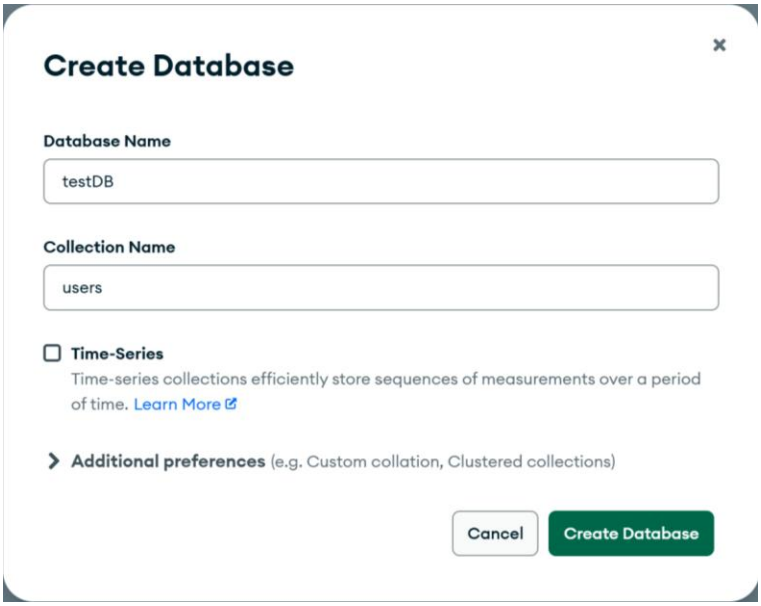
📁 Step 1: Create a Database



📁 Step 2: Create a Collection and Insert Data

```
>_MONGOSH
> use testDB
< switched to db testDB
> db.users.insertOne({
    name: "Alice",
    age: 25,
    email: "alice@example.com"
  })
< {
    acknowledged: true,
    insertedId: ObjectId('6acdc301b9db0e4acc9ec9cd')
  }
> db.users.insertMany([
    { name: "Bob", age: 30, email: "bob@example.com" },
    { name: "Charlie", age: 28, email: "charlie@example.com" }
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('6acdc318b9db0e4acc9ec9ce'),
      '1': ObjectId('6acdc318b9db0e4acc9ec9cf')
    }
  }
```

🔍 Step 3: Query the Collection

```
>_MONGOSH
> db.users.find()
< {
    _id: ObjectId('6acdc301b9db0e4acc9ec9cd'),
    name: 'Alice',
    age: 25,
    email: 'alice@example.com'
  }
  {
    _id: ObjectId('6acdc318b9db0e4acc9ec9ce'),
    name: 'Bob',
    age: 30,
    email: 'bob@example.com'
  }
  {
    _id: ObjectId('6acdc318b9db0e4acc9ec9cf'),
    name: 'Charlie',
    age: 28,
    email: 'charlie@example.com'
  }
```

```
> db.users.find({ age: { $gt: 27 } })
< {
    _id: ObjectId('6acdc318b9db0e4acc9ec9ce'),
    name: 'Bob',
    age: 30,
    email: 'bob@example.com'
  }
  {
    _id: ObjectId('6acdc318b9db0e4acc9ec9cf'),
    name: 'Charlie',
    age: 28,
    email: 'charlie@example.com'
  }
```

📝 Step 4: Update a Document

```
> db.users.updateOne(
    { name: "Alice" },
    { $set: { age: 26 } }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

❌ Step 5: Delete a Document

```
> db.users.deleteOne({ name: "Bob" })
< {
    acknowledged: true,
    deletedCount: 1
  }
```

Step 6: Check All Documents

```
> db.users.find().pretty()
< {
    _id: ObjectId('6acdc301b9db0e4acc9ec9cd'),
    name: 'Alice',
    age: 26,
    email: 'alice@example.com'
  }
  {
    _id: ObjectId('6acdc318b9db0e4acc9ec9cf'),
    name: 'Charlie',
    age: 28,
    email: 'charlie@example.com'
  }
```

✅ Example 1: Working with Embedded Documents

📁 Task: Insert and query documents with nested data

```
>_MONGOSH
> db.orders.insertMany([
    {
      order_id: 1001,
      customer: "Aung Aung",
      items: [
        { product: "Pen", quantity: 2, price: 500 },
        { product: "Notebook", quantity: 1, price: 1000 }
      ],
      order_date: ISODate("2024-05-01")
    },
    {
      order_id: 1002,
      customer: "Mya Mya",
      items: [
        { product: "Eraser", quantity: 3, price: 200 }
      ],
      order_date: ISODate("2024-05-03")
    }
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('6acddbf8b9db0e4acc9ec9d4'),
      '1': ObjectId('6acddbf8b9db0e4acc9ec9d5')
    }
  }
```

🔍 **Query: Find all orders with a product = "Pen"**

```
>_MONGOSH
> db.orders.find({ "items.product": "Pen" })
< {
    _id: ObjectId('6acddbf8b9db0e4acc9ec9d4'),
    order_id: 1001,
    customer: 'Aung Aung',
    items: [
      {
        product: 'Pen',
        quantity: 2,
        price: 500
      },
      {
        product: 'Notebook',
        quantity: 1,
        price: 1000
      }
    ],
    order_date: 2024-05-01T00:00:00.000Z
  }
```

✅ **Example 2: Aggregation – Total Sales per Customer**

📁 **Task: Use aggregation to calculate total amount per customer**

```
>_MONGOSH
> db.orders.aggregate([
    { $unwind: "$items" },
    {
      $group: {
        _id: "$customer",
        total_spent: { $sum: { $multiply: ["$items.quantity", "$items.price"] } }
      }
    }
  ])
< {
    _id: 'Aung Aung',
    total_spent: 2000
  }
  {
    _id: 'Mya Mya',
    total_spent: 600
  }
```

✅ **Example 3: Update Array – Push a new item into an order**

📁 **Task: Add a new item to an existing order**

```
> db.orders.updateOne(
    { order_id: 1002 },
    {
      $push: {
        items: { product: "Ruler", quantity: 1, price: 700 }
      }
    }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

✅ Example 4: Create Index for Fast Search

📁 Task: Add an index on customer field

```
>_MONGOSH
> db.orders.createIndex({ customer: 1 })
< customer_1
> db.orders.find({ customer: "Aung Aung" }).explain("executionStats")
< {
    explainVersion: '1',
    queryPlanner: {
      namespace: 'testDB.orders',
      parsedQuery: {
        customer: {
          '$eq': 'Aung Aung'
        }
      },
      indexFilterSet: false,
      queryHash: '6D782773',
      planCacheShapeHash: '6D782773',
      planCacheKey: '4D4758CB',
      optimizationTimeMillis: 3,
      maxIndexedOrSolutionsReached: false,
      maxIndexedAndSolutionsReached: false,
      maxScansToExplodeReached: false,
      prunedSimilarIndexes: false,
      winningPlan: {
        isCached: false,
        stage: 'FETCH',
        nss: 'testDB.orders',
        inputStage: {
          stage: 'IXSCAN',
          nss: 'testDB.orders',
          keyPattern: {
```

✅ Example 5: Use $lookup for Join-like Operation

🧱 Add a customer collection:

```
>_MONGOSH
> db.customers.insertMany([
    { name: "Aung Aung", city: "Yangon" },
    { name: "Mya Mya", city: "Mandalay" }
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('6acddc33b9db0e4acc9ec9d6'),
      '1': ObjectId('6acddc33b9db0e4acc9ec9d7')
    }
  }
```

🔄 Join orders with customer info:

```
>_MONGOSH
> db.orders.aggregate([
    {
      $lookup: {
        from: "customers",
        localField: "customer",
        foreignField: "name",
        as: "customer_info"
      }
    }
  ])
< {
    _id: ObjectId('6acddbf8b9db0e4acc9ec9d4'),
    order_id: 1001,
    customer: 'Aung Aung',
    items: [
      {
        product: 'Pen',
        quantity: 2,
        price: 500
      },
      {
        product: 'Notebook',
        quantity: 1,
        price: 1000
      }
    ],
    order_date: 2024-05-01T00:00:00.000Z,
    customer_info: [
      {
```

Employee.json

1. List all the employees

```
>_MONGOSH
> db.employees.find()
< {
    _id: ObjectId('6acde0c5fbf6d8296f92c6cd'),
    name: 'Steve',
    age: 35,
    phone: {
      ext: '1234',
      personal: '555-0101'
    },
    favorites: {
      food: 'burgers',
      artist: 'Van Gogh'
    },
    privileges: 'user'
  }
  {
    _id: ObjectId('6acde0c5fbf6d8296f92c6ce'),
    name: 'Alice',
    age: 28,
    phone: {
      ext: '2143',
      personal: '555-0202'
    },
    favorites: {
      food: 'pizza',
      artist: 'Monet'
    },
    privileges: 'user'
  }
  {
    _id: ObjectId('6acde0c5fbf6d8296f92c6cf'),
    name: 'Willy',
    age: 25,
    phone: {
      ext: '4321',
      personal: '555-0303'
    },
    favorites: {
      food: 'sushi',
      artist: 'Picasso'
    },
    privileges: 'user'
  }
  {
    _id: ObjectId('6acde0c5fbf6d8296f92c6d0'),
    name: 'Bob',
    age: 42,
    phone: {
      ext: '5678',
      personal: '555-0404'
    },
    favorites: {
      food: 'pizza',
      artist: 'Da Vinci'
    },
    privileges: 'admin'
  }
```

2. Find the employee whose name is Steve

```
> db.employees.findOne({ name: "Steve" })
< {
    _id: ObjectId('6acde0c5fbf6d8296f92c6cd'),
    name: 'Steve',
    age: 35,
    phone: {
      ext: '1234',
      personal: '555-0101'
    },
    favorites: {
      food: 'burgers',
      artist: 'Van Gogh'
    },
    privileges: 'user'
  }
```

3. Find all employees whose age is greater than 30

```
>_MONGOSH
> db.employees.find({ age: { $gt: 30 } })
< {
    _id: ObjectId('6acde0c5fbf6d8296f92c6cd'),
    name: 'Steve',
    age: 35,
    phone: {
      ext: '1234',
      personal: '555-0101'
    },
    favorites: {
      food: 'burgers',
      artist: 'Van Gogh'
    },
    privileges: 'user'
  }
  {
    _id: ObjectId('6acde0c5fbf6d8296f92c6d0'),
    name: 'Bob',
    age: 42,
    phone: {
      ext: '5678',
      personal: '555-0404'
    },
    favorites: {
      food: 'pizza',
      artist: 'Da Vinci'
    },
    privileges: 'admin'
  }
  {
    _id: ObjectId('6acde0c5fbf6d8296f92c6d1'),
    name: 'John',
    age: 31,
    phone: {
      ext: '9999',
      personal: '555-0505'
    },
    favorites: {
      food: 'tacos',
      artist: 'Dalí'
    },
    privileges: 'user'
  }
```

4. Find the employee whose extension is 2143

```
> db.employees.findOne({ "phone.ext": "2143" })
< {
    _id: ObjectId('6acde0c5fbf6d8296f92c6ce'),
    name: 'Alice',
    age: 28,
    phone: {
      ext: '2143',
      personal: '555-0202'
    },
    favorites: {
      food: 'pizza',
      artist: 'Monet'
    },
    privileges: 'user'
  }
```

5. Find all employees that are over 30

(Same as query #3)

db.employees.find({ age: { $gt: 30 } })

6. Find all employees that are less than or equal to 30

```
>_MONGOSH
> db.employees.find({ age: { $lte: 30 } })
< {
    _id: ObjectId('6acde0c5fbf6d8296f92c6ce'),
    name: 'Alice',
    age: 28,
    phone: {
      ext: '2143',
      personal: '555-0202'
    },
    favorites: {
      food: 'pizza',
      artist: 'Monet'
    },
    privileges: 'user'
  }
```

```
{
  _id: ObjectId('6acde0c5fbf6d8296f92c6cf'),
  name: 'Willy',
  age: 25,
  phone: {
    ext: '4321',
    personal: '555-0303'
  },
  favorites: {
    food: 'sushi',
    artist: 'Picasso'
  },
  privileges: 'user'
}
```

7. Find all the employees whose favorite food is pizza

```
>_MONGOSH
> db.employees.find({ "favorites.food": "pizza" })
< {
    _id: ObjectId('6acde0c5fbf6d8296f92c6ce'),
    name: 'Alice',
    age: 28,
    phone: {
      ext: '2143',
      personal: '555-0202'
    },
    favorites: {
      food: 'pizza',
      artist: 'Monet'
    },
    privileges: 'user'
  }
  {
    _id: ObjectId('6acde0c5fbf6d8296f92c6d0'),
    name: 'Bob',
    age: 42,
    phone: {
      ext: '5678',
      personal: '555-0404'
    },
    favorites: {
      food: 'pizza',
      artist: 'Da Vinci'
    },
    privileges: 'admin'
  }
```

8. Change Willy's personal phone number to "93-123-45-67"

```
> db.employees.updateOne(
    { name: "Willy" },
    { $set: { "phone.personal": "93-123-45-67" } }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

9. Change Bob's privilege to normal user

```
> db.employees.updateOne(
    { name: "Bob" },
    { $set: { privileges: "user" } }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

10. Find all employees whose favorite artist is equal to Picasso

```
> db.employees.find({ "favorites.artist": "Picasso" })
< {
    _id: ObjectId('6acde0c5fbf6d8296f92c6cf'),
    name: 'Willy',
    age: 25,
    phone: {
      ext: '4321',
      personal: '93-123-45-67'
    },
    favorites: {
      food: 'sushi',
      artist: 'Picasso'
    },
    privileges: 'user'
  }
```

11. Delete the user John

```
> db.employees.deleteOne({ name: "John" })
< {
    acknowledged: true,
    deletedCount: 1
  }
```

== restaurants.json ==

🔍 Find Queries

// 1. List all the restaurants



// 2. Find all restaurants and display only specific fields

```
>_MONGOSH
> db.restaurants.find({}, {restaurant_id: 1, name: 1, borough: 1, cuisine: 1})
< {
    _id: ObjectId('6acde5d6fbf6d8296f92c6d5'),
    restaurant_id: '1001',
    name: 'Bronx Grill',
    borough: 'Bronx',
    cuisine: 'Steak'
  }
  {
    _id: ObjectId('6acde5d6fbf6d8296f92c6d6'),
    restaurant_id: '1002',
    name: 'Brooklyn Steakhouse',
    borough: 'Brooklyn',
    cuisine: 'Steak'
  }
  {
    _id: ObjectId('6acde5d6fbf6d8296f92c6d7'),
    restaurant_id: '1003',
    name: 'Morris Park Bake Shop',
    borough: 'Bronx',
    cuisine: 'Bakery'
  }
  {
    _id: ObjectId('6acde5d6fbf6d8296f92c6d8'),
    restaurant_id: '1004',
    name: 'Classic Diner',
    borough: 'Queens',
    cuisine: 'American '
  }
```

// 3. Display restaurant_id, name, borough, and zip code only

```
>_MONGOSH
> db.restaurants.find({}, {
    restaurant_id: 1,
    name: 1,
    borough: 1,
    "address.zipcode": 1
  })
< {
    _id: ObjectId('6acde5d6fbf6d8296f92c6d5'),
    restaurant_id: '1001',
    name: 'Bronx Grill',
    borough: 'Bronx',
    address: {
      zipcode: '10466'
    }
  }
  {
    _id: ObjectId('6acde5d6fbf6d8296f92c6d6'),
    restaurant_id: '1002',
    name: 'Brooklyn Steakhouse',
    borough: 'Brooklyn',
    address: {
      zipcode: '11201'
    }
  }
  {
    _id: ObjectId('6acde5d6fbf6d8296f92c6d7'),
    restaurant_id: '1003',
    name: 'Morris Park Bake Shop',
    borough: 'Bronx',
```

// 4. Restaurants in the borough Bronx

```
>_MONGOSH
> db.restaurants.find({borough: "Bronx"})
< {
    _id: ObjectId('6acde5d6fbf6d8296f92c6d5'),
    restaurant_id: '1001',
    name: 'Bronx Grill',
    borough: 'Bronx',
    cuisine: 'Steak',
    address: {
      street: 'Main St',
      zipcode: '10466'
    },
    grades: [
      {
        score: 95,
        grade: 'A'
      }
    ]
  }
  {
    _id: ObjectId('6acde5d6fbf6d8296f92c6d7'),
    restaurant_id: '1003',
    name: 'Morris Park Bake Shop',
    borough: 'Bronx',
    cuisine: 'Bakery',
    address: {
      street: 'Original St',
      zipcode: '10462'
    },
    grades: [
```

// 5. Brooklyn restaurants with Steak cuisine

```
> db.restaurants.find({
    borough: "Brooklyn",
    cuisine: "Steak"
  })
< {
    _id: ObjectId('6acde5d6fbf6d8296f92c6d6'),
    restaurant_id: '1002',
    name: 'Brooklyn Steakhouse',
    borough: 'Brooklyn',
    cuisine: 'Steak',
    address: {
      street: 'Atlantic Ave',
      zipcode: '11201'
    },
    grades: [
      {
        score: 80,
        grade: 'A'
      }
    ]
  }
```

// 6. Restaurants with a score > 90

```
>_MONGOSH
> db.restaurants.find({"grades.score": {$gt: 90}})
< {
    _id: ObjectId('6acde5d6fbf6d8296f92c6d5'),
    restaurant_id: '1001',
    name: 'Bronx Grill',
    borough: 'Bronx',
    cuisine: 'Steak',
    address: {
      street: 'Main St',
      zipcode: '10466'
    },
    grades: [
      {
        score: 95,
        grade: 'A'
      }
    ]
  }
  {
    _id: ObjectId('6acde5d6fbf6d8296f92c6d9'),
    restaurant_id: '1005',
    name: 'Golden Dragon',
    borough: 'Manhattan',
    cuisine: 'Chinese',
    address: {
      street: 'Broadway',
      zipcode: '10001'
    },
    grades: [
```

// 7. Non-Bakery restaurants with score >= 70

```
>_MONGOSH
> db.restaurants.find({
    cuisine: {$ne: "Bakery"},
    "grades.score": {$gte: 70}
  })
< {
    _id: ObjectId('6acde5d6fbf6d8296f92c6d5'),
    restaurant_id: '1001',
    name: 'Bronx Grill',
    borough: 'Bronx',
    cuisine: 'Steak',
    address: {
      street: 'Main St',
      zipcode: '10466'
    },
    grades: [
      {
        score: 95,
        grade: 'A'
      }
    ]
  }
  {
    _id: ObjectId('6acde5d6fbf6d8296f92c6d6'),
    restaurant_id: '1002',
    name: 'Brooklyn Steakhouse',
```

// 8. Non-Chinese, grade A, not in Manhattan

```
>_MONGOSH
> db.restaurants.find({
    cuisine: {$ne: "Chinese"},
    "grades.grade": "A",
    borough: {$ne: "Manhattan"}
  })
< {
    _id: ObjectId('6acde5d6fbf6d8296f92c6d5'),
    restaurant_id: '1001',
    name: 'Bronx Grill',
    borough: 'Bronx',
    cuisine: 'Steak',
    address: {
      street: 'Main St',
      zipcode: '10466'
    },
    grades: [
      {
        score: 95,
        grade: 'A'
      }
    ]
  }
  {
    _id: ObjectId('6acde5d6fbf6d8296f92c6d6'),
    restaurant_id: '1002',
    name: 'Brooklyn Steakhouse',
```

🛠️ Update Commands

// 9. Update 'American ' to 'American' (note the space)

```
> db.restaurants.updateMany(
    {cuisine: "American "},
    {$set: {cuisine: "American"}}
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

// 10. Update Morris Park Bake Shop street address

```
> db.restaurants.updateOne(
    {name: "Morris Park Bake Shop"},
    {$set: {"address.street": "Calle falsa 123"}}
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

🗑️ Delete Command

// 11. Delete all restaurants with zipcode 10466

```
> db.restaurants.deleteMany({"address.zipcode": "10466"})
< {
    acknowledged: true,
    deletedCount: 1
  }
```

== companies.json ==

1. **Find all the companies that include 'Facebook' on the name field**

```
> db.companies.find({ name: /Facebook/ })
< {
    _id: ObjectId('6ace0954fbf6d8296f92c6db'),
    name: 'Facebook',
    category_code: 'social',
    founded_year: 2004,
    founded_month: 'February',
    founded_day: 4,
    number_of_employees: 50000
  }
```

2. **Find all the companies which category_code is 'web'. Retrieve only their name field**

```
> db.companies.find(
    { category_code: 'web' },
    { name: 1, _id: 0 }
  )
< {
    name: 'Webify Solutions'
  }
```

3. **Find all the companies named "Twitter", and retrieve only their name, category_code and founded_year fields**

```
> db.companies.find(
    { name: "Twitter" },
    { name: 1, category_code: 1, founded_year: 1, _id: 0 }
  )
< {
    name: 'Twitter',
    category_code: 'social',
    founded_year: 2006
  }
```

4. **Find all the companies who have web as their category_code, but limit the search to 50 companies**

```
> db.companies.find(
    { category_code: 'web' }
  ).limit(50)
< {
    _id: ObjectId('6ace0954fbf6d8296f92c6dc'),
    name: 'Webify Solutions',
    category_code: 'web',
    founded_year: 2005,
    founded_month: 'May',
    founded_day: 12,
    number_of_employees: 50
  }
```

5. **Find all the companies which category_code is 'enterprise' and have been founded in 2005. Retrieve only the name, category_code and founded_year fields**

```
> db.companies.find(
    { category_code: 'enterprise', founded_year: 2005 },
    { name: 1, category_code: 1, founded_year: 1, _id: 0 }
  )
< {
    name: 'Enterprise Logic',
    category_code: 'enterprise',
    founded_year: 2005
  }
```

6. **Find all the companies that have been founded in 2000 or have 20 employees. Sort them descendingly by number_of_employees**

```
> db.companies.find(
    { $or: [{ founded_year: 2000 }, { number_of_employees: 20 }] }
  ).sort({ number_of_employees: -1 })
< {
    _id: ObjectId('6ace0954fbf6d8296f92c6df'),
    name: 'Millennium Tech',
    category_code: 'software',
    founded_year: 2000,
    founded_month: 'June',
    founded_day: 15,
    number_of_employees: 100
  }
  {
    _id: ObjectId('6ace0954fbf6d8296f92c6e0'),
    name: 'Midsize Corp',
    category_code: 'hardware',
    founded_year: 1998,
    founded_month: 'July',
    founded_day: 1,
    number_of_employees: 20
  }
```

7. **Find all the companies that do not include web nor social on their category_code. Limit the search to 20 documents and retrieve only their name and category_code**

```
> db.companies.find(
    { category_code: { $nin: ['web', 'social'] } },
    { name: 1, category_code: 1, _id: 0 }
  ).limit(20)
< {
    name: 'Enterprise Logic',
    category_code: 'enterprise'
  }
  {
    name: 'Millennium Tech',
    category_code: 'software'
  }
  {
    name: 'Midsize Corp',
    category_code: 'hardware'
  }
  {
    name: 'Acme Innovations',
    category_code: 'biotech'
  }
  {
    name: 'Alpha Ventures',
    category_code: 'finance'
  }
  {
    name: 'Beta Systems',
    category_code: 'software'
  }
```

8. **Find all the companies that were not founded on 'June'. Skip the first 50 results and retrieve only the founded_month and name fields**

```
> db.companies.find(
    { founded_month: { $ne: 'June' } },
    { founded_month: 1, name: 1, _id: 0 }
  ).skip(50)
<
testDB >
```

9. **Find all the companies that have 50 employees, but do not correspond to the 'web' category_code**

```
> db.companies.find(
    { number_of_employees: 50, category_code: { $ne: 'web' } }
  )
<
testDB >
```

10. **Find all the companies that have been founded on the 1st of the month, but do not have either 50 employees nor 'web' as their category_code. Retrieve only the founded_day and name and limit the search to 5 documents**

```
> db.companies.find(
    {
      founded_day: 1,
      number_of_employees: { $ne: 50 },
      category_code: { $ne: 'web' }
    },
    { founded_day: 1, name: 1, _id: 0 }
  ).limit(5)
< {
    name: 'Enterprise Logic',
    founded_day: 1
  }
```

```
{
  name: 'Midsize Corp',
  founded_day: 1
}
{
  name: 'Acme Innovations',
  founded_day: 1
}
{
  name: 'Alpha Ventures',
  founded_day: 1
}
```

11. **Find all the companies which the price_amount of the acquisition was 40,000,000. Sort them by name**

```
> db.companies.find(
    { "acquisition.price_amount": 40000000 }
  ).sort({ name: 1 })
< {
    _id: ObjectId('6ace0954fbf6d8296f92c6e2'),
    name: 'Alpha Ventures',
    category_code: 'finance',
    founded_year: 2012,
    founded_month: 'January',
    founded_day: 1,
    number_of_employees: 40,
    acquisition: {
      price_amount: 40000000,
      acquired_month: 5,
      acquired_year: 2015
    }
  }
```

12. **Find all the companies that have been acquired on January of 2014. Retrieve only the acquisition and name fields**

```
> db.companies.find(
    {
      "acquisition.acquired_month": 1,
      "acquisition.acquired_year": 2014
    },
    { acquisition: 1, name: 1, _id: 0 }
  )
< {
    name: 'Beta Systems',
    acquisition: {
      price_amount: 15000000,
      acquired_month: 1,
      acquired_year: 2014
    }
  }
```